Web Application Penetration Test Report

OWASP Juice Shop

IDOR · Negative Quantity · SQL Injection

Prepared by	Rahaf Albarakati
Test date	October 2026
Target	OWASP Juice Shop — http://localhost:3000
Test type	Gray box, using two standard user accounts
Tools	Burp Suite Community 2026.10.1, Chrome

> This test was performed on a local instance of OWASP Juice Shop, an application intentionally built with vulnerabilities for learning purposes. No real system was tested, and all accounts and data shown in the evidence are test data.

Executive Summary

Three vulnerabilities were confirmed with evidence. The most severe is a SQL injection on the login page that allows anyone — without an account — to log in with administrator privileges. A fourth hypothesis (XSS in the search field) was tested and could not be proven with the available evidence.

#	Vulnerability	Test point	Severity
1	Authentication bypass via SQL injection	`POST /rest/user/login`	Critical
2	Access to another user's basket (IDOR)	`GET /rest/basket/:id`	High
3	Negative quantity accepted, producing a negative total	`PUT /api/BasketItems/:id`	High
—	XSS in the search field	Search field	Not proven

Testing Methodology

Each case followed the same steps: define the **expected behavior**, identify the **observation** that raised suspicion, form a testable **hypothesis**, **test** it by changing a single variable, and finally collect the **evidence** that confirms or rejects the hypothesis.

Severity Rating Methodology

Each vulnerability is rated on two separate axes, and the final rating is derived from both:

Axis	What it measures
Ease of exploitation	Does the attacker need an account? Does it require special skills or tools? Is the condition always present?
Impact	Which data or privileges are affected? Does it affect one user or the whole system? Is there financial loss?

Finding 1

Critical

Authentication Bypass via SQL Injection

Test point	`POST /rest/user/login`
Classification	OWASP A03:2021 – Injection · CWE-89 · CWE-287

The email field is concatenated directly into the database query, so a simple payload grants access to the administrator account without a password.

1. **Expected behavior:** Login succeeds only when the email matches the correct password.
2. **Observation:** Entering a single quote `'` in the email field affected the query's behavior.
3. **Hypothesis:** If the query is built through string concatenation, an always-true condition will disable the password check and return the first user in the table (the administrator).
4. **Test:** Entered the payload `' OR 1=1--` in the email field with a random password.

Evidence

```
POST /rest/user/login HTTP/1.1
{"email":"' OR 1=1--","password":"<random>"}

HTTP/1.1 200 OK
{"authentication":{"token":"eyJ0eXAi…","bid":1,"umail":"admin@juice-sh.op"}}
```

The server returned 200 OK with a valid session token tied to the administrator account.

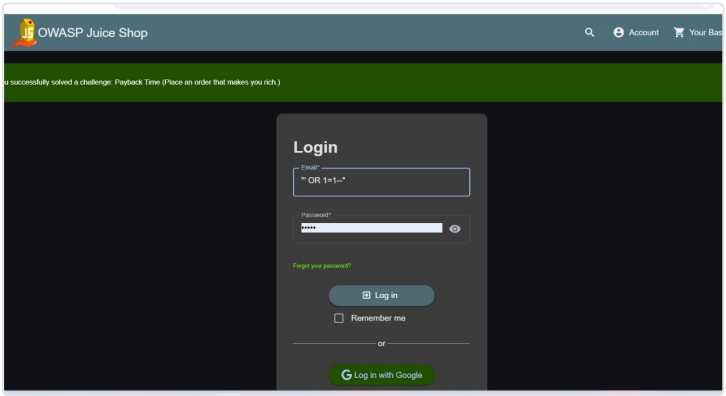


Figure 1: Payload in the email field

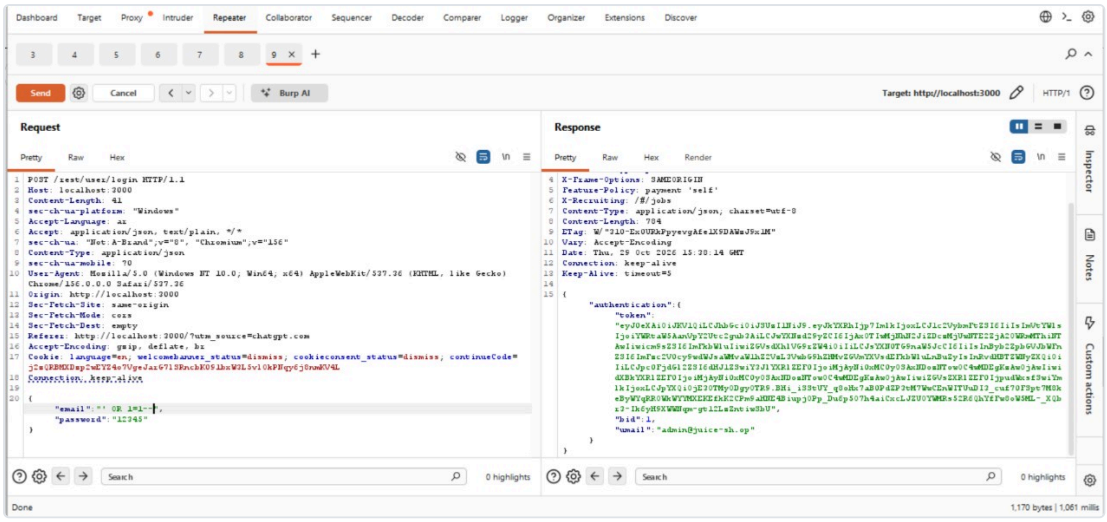


Figure 2: Request and response in Burp — token for admin@juice-sh.op

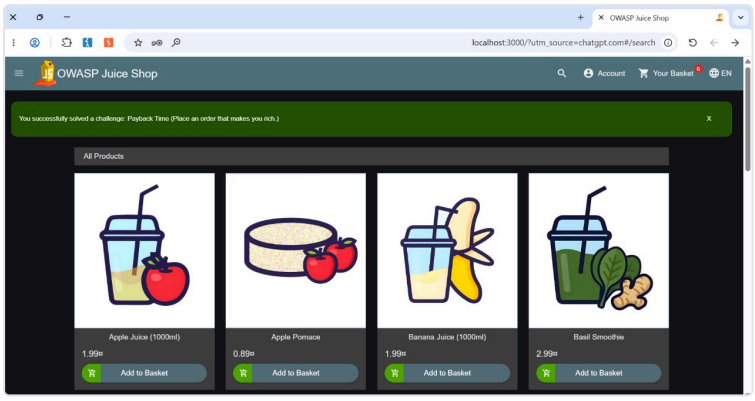


Figure 3: Active session with administrator privileges after the bypass

Severity Rating

Ease of exploitation	Impact	Final rating
Very high	**Critical**	**Critical**
Remote, no account required, a well-known one-line payload.	Takeover of the highest-privileged account, threatening the confidentiality and integrity of the whole system.	Highest ease combined with highest impact.

The test did not include extracting additional data from the database, so this is not claimed as a confirmed result.

Recommendation

- Use parameterized queries instead of concatenating user input into the query string.
- Validate the email format on the server before using it.
- Return generic error messages that do not reveal database details.
- Restrict the database account's privileges to the minimum required.

Finding 2

High

Access to Another User's Basket (IDOR)

Test point	`GET /rest/basket/:id`
Classification	OWASP A01:2021 – Broken Access Control · CWE-639

The server does not verify basket ownership, so any logged-in user can read another user's basket by changing its number in the URL.

1. **Expected behavior:** Each user can only see their own basket.
2. **Observation:** The basket ID is a sequential number in the path (`/rest/basket/6`). This alone does not prove a vulnerability, but it warrants testing.
3. **Hypothesis:** The server may return the basket for any ID without checking that it belongs to the token owner.
4. **Test:** Changed the ID from 6 to 7 while keeping account A's token unchanged, then compared the response with account B's data.

Evidence

Request	Token used	Response	Result
`GET /rest/basket/6`	Account A	UserId = 25	Normal
`GET /rest/basket/7`	Account B	UserId = 26	Baseline
`GET /rest/basket/7`	**Account A**	**UserId = 26**	**Confirmed**

```
# Authorization: Bearer <token A>
GET /rest/basket/7 HTTP/1.1

HTTP/1.1 200 OK
{"status":"success","data":{"id":7,"coupon":null,"UserId":26,"Products":[]}}
```

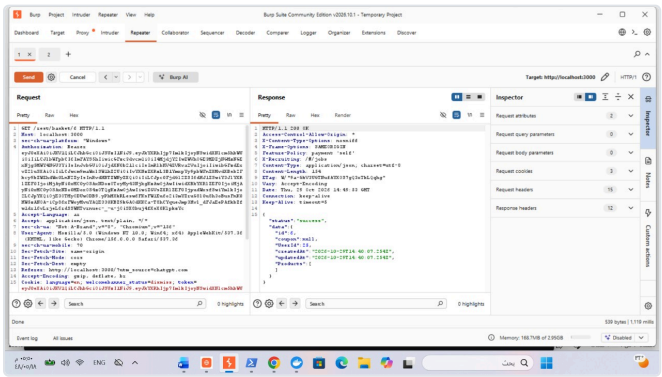


Figure 4: Account A requests its own basket (bid=6) — response UserId=25

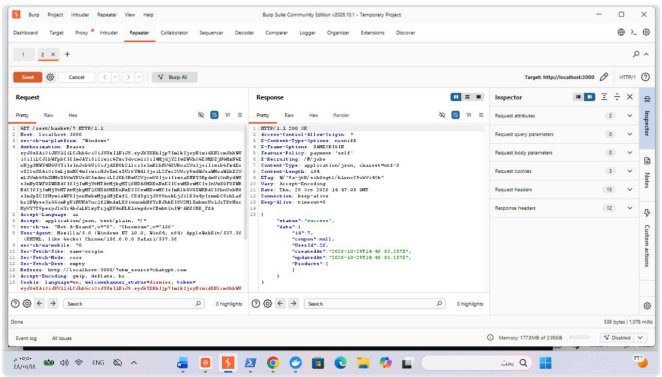


Figure 5: Baseline — account B's basket (bid=7) with its own token — UserId=26

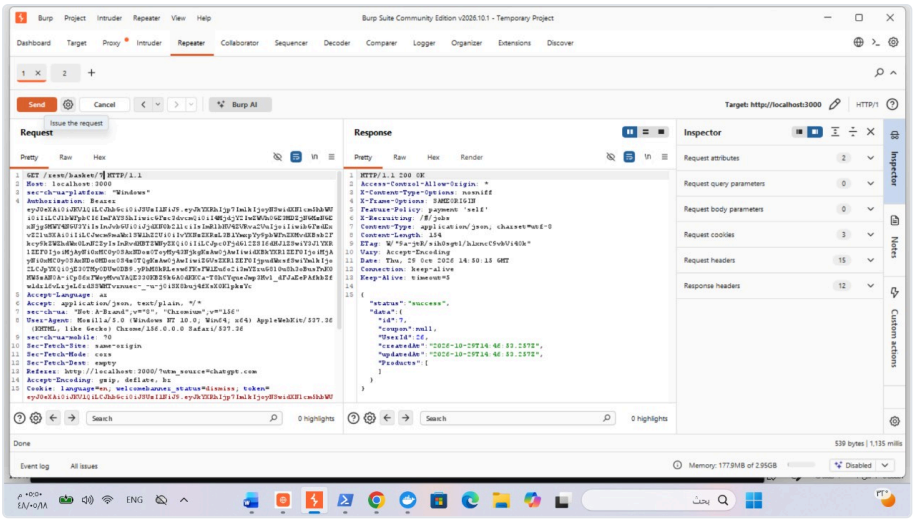


Figure 6: Exploitation — account A's token returns account B's basket (UserId=26)

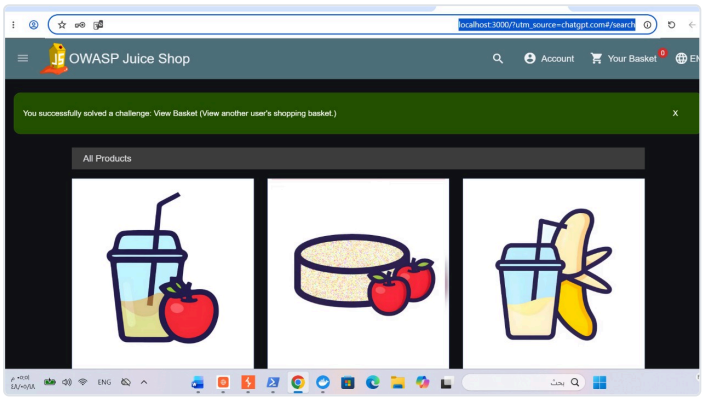


Figure 7: The application confirms the "View Basket" challenge was solved

Severity Rating

Ease of exploitation	Impact	Final rating
High	**Medium**	**High**
Any standard account and a guessable sequential number are enough.	Read-only access to basket contents, but horizontally across all users.	High ease with an impact that covers every user.

Modifying other users' baskets or orders was not tested; it is recommended to test this, as the same weak ownership-check pattern may repeat in other operations.

Recommendation

- On every request, verify on the server that the requested basket belongs to the token owner.
- Preferably, load the user's basket from the session identity instead of accepting its ID from the client.
- Reject unauthorized requests with 403 Forbidden or 404 Not Found without revealing any data.

Finding 3

High

Negative Quantity Accepted, Producing a Negative Total

Test point	`PUT /api/BasketItems/:id`
Classification	OWASP A04:2021 – Insecure Design · CWE-20 · CWE-840

The server does not validate the sign of the quantity, so it accepts a negative value and completes the order with a negative total — meaning the shop owes the customer instead of the other way around.

1. **Expected behavior:** Quantity is a positive integer (1 or more), and the total is never below zero.
2. **Observation:** The quantity is sent by the client in the request body (`{"quantity":3}`) and accepted as-is, even though it controls the financial calculation.
3. **Hypothesis:** If the server does not validate the sign, a negative value will be accepted and produce a negative total.
4. **Test:** Intercepted the quantity update request in Burp, changed the quantity from 3 to -3, then completed the checkout.

Evidence

```
PUT /api/BasketItems/{id} HTTP/1.1
{"quantity":-3}

HTTP/1.1 200 OK
{"status":"success","data":{…,"quantity":-3,…}}
```

	Quantity	Items total	Final total
Before tampering	3	5.97	5.97
After tampering	**-3**	**-5.97**	**-4.98**

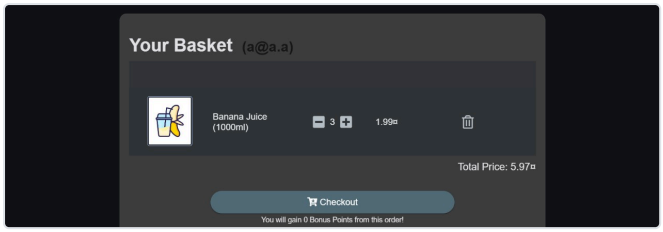


Figure 8: Basket in its normal state (quantity 3, total 5.97)

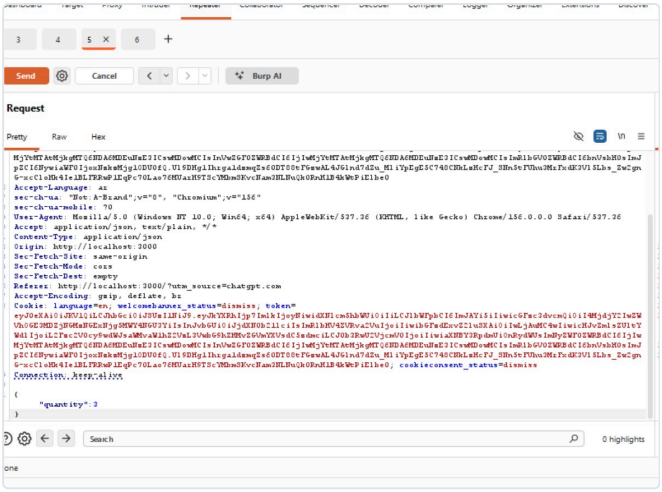


Figure 9: Normal quantity update request

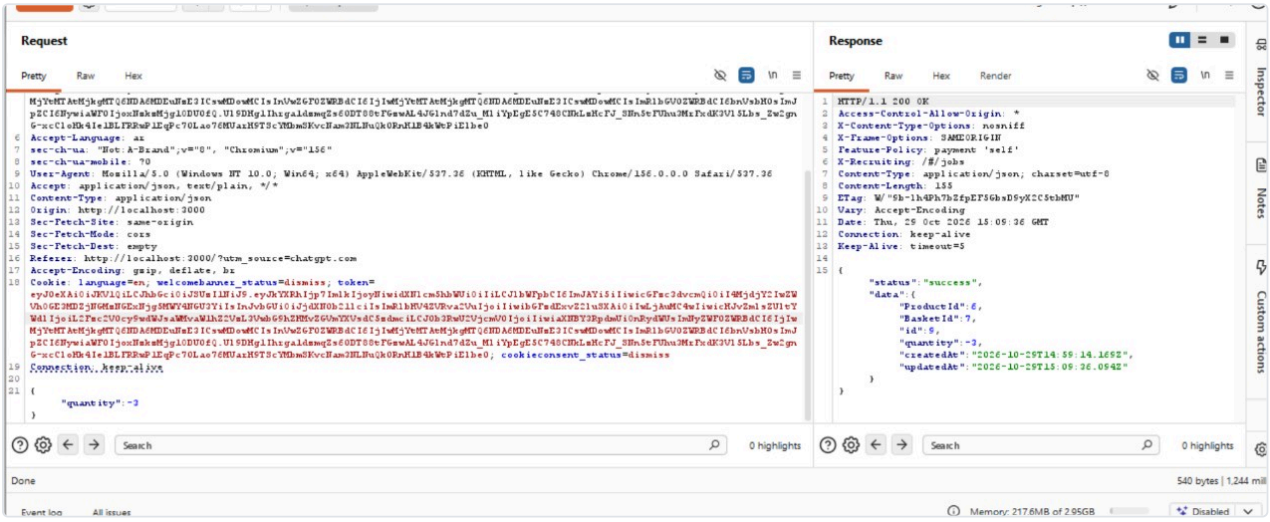


Figure 10: Request with quantity -3 — the server accepts and stores the value

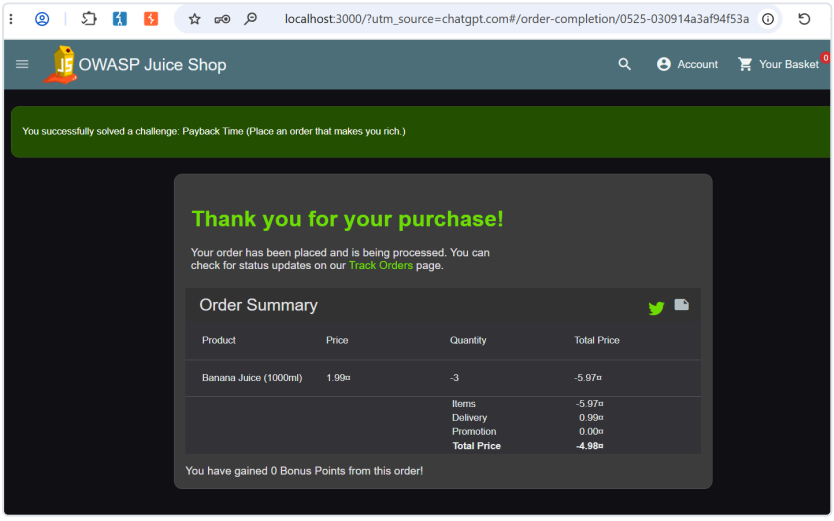


Figure 11: Order summary with a total of -4.98 and the "Payback Time" challenge confirmation

Severity Rating

Ease of exploitation	Impact	Final rating
High	**High**	**High**
Any user with an account, by modifying a single value in the request.	Direct financial loss and broken billing logic, though limited to orders and not affecting user data.	Direct financial damage that is easy to repeat.

Recommendation

- Validate the quantity on the server: a positive integer within a reasonable maximum, rejecting any value below 1.
- Recalculate the total on the server and prevent creating any order with a negative total.

Hypothesis Tested — No Vulnerability Proven

Possible XSS in the Search Field

Not proven

Hypothesis

The search field may allow JavaScript code to be entered and executed in the browser, which could lead to a Cross-Site Scripting (XSS) vulnerability.

What did I test?

I entered a test value containing JavaScript code into the search field, then observed the application's behavior and the result shown on the page.

Result

The search worked, but no JavaScript alert appeared, and I could not prove that the code executed in the browser.

Why was the hypothesis dismissed?

This case was excluded from the list of confirmed vulnerabilities because the test showed no clear evidence of JavaScript execution or any real security impact. The search functioning on its own does not prove an XSS vulnerability.

> The hypothesis was therefore recorded as **not proven**, not as **definitively secure**.